

Technical Requirement Document

Here is the Technical Requirement Document (TRD) based on the provided Functional Requirement Document (FRD):

TR-DLT-001: Load Customer and Order Data into Delta Tables

Related Functional Requirement(s): FR-DLT-001

Objective: Implement a Delta Live Tables pipeline to load customer and order data from CSV files into Delta tables and perform data cleansing.

Target Cluster Configuration:

Cluster Name: Delta Live Tables Cluster

Databricks Runtime Version: 10.4.x-scala2.12

Node Type: Standard_DS3_v2

Driver Node: Standard_DS3_v2

Worker Nodes: 2-4 nodes with autoscaling

Autoscaling: Enabled

Auto Termination: 30 minutes

Libraries Installed: delta-lake, spark-csv

Source Data Details:

Customer CSV data: `/Volumes/gen\_ai\_poc\_databrickscoe/sdlc\_wizard/customerdata`, CSV format

Order CSV data: `/Volumes/gen\_ai\_poc\_databrickscoe/sdlc\_wizard/orderdata`, CSV format

Target Data Details:

Customer Delta table: "customer", Delta format

Order Delta table: "order", Delta format

Job Flow / Pipeline Stages:

- Read customer CSV data and load into "customer" Delta table
- Read order CSV data and load into "order" Delta table
- Add "TotalAmount" column to "order" table

- Remove null and duplicate records from both tables

Data Transformations / Business Logic:

Step 1: Read customer CSV data and load into "customer" Delta table

- Transformation Logic: Use ``spark.read.csv()`` to read CSV data and ``write.format("delta").saveAsTable()`` to write to Delta table

Step 2: Read order CSV data and load into "order" Delta table

- Transformation Logic: Use ``spark.read.csv()`` to read CSV data and ``write.format("delta").saveAsTable()`` to write to Delta table

Step 3: Add "TotalAmount" column to "order" table

- Transformation Logic: Use ``withColumn()`` to add new column and calculate value using ``PricePerUnit`` and ``Qty`` columns

Step 4: Remove null and duplicate records from both tables

- Transformation Logic: Use ``dropna()`` and ``dropDuplicates()`` to remove null and duplicate records

Error Handling and Logging:

- Log errors and exceptions to a log file or monitoring system
- Use try-except blocks to catch and handle exceptions during data processing

TR-DLT-002: Create and Load ordersummary Table

Related Functional Requirement(s): FR-DLT-002

Objective: Implement a Delta Live Tables pipeline to create and load "ordersummary" table with joined data from "customer" and "order" tables using SCD Type 2.

Target Cluster Configuration: Same as TR-DLT-001

Source Data Details:

Customer Delta table: "customer", Delta format

Order Delta table: "order", Delta format

Target Data Details:

ordersummary Delta table: "ordersummary", Delta format

Job Flow / Pipeline Stages:

- Create "ordersummary" table if it does not exist
- Join "customer" and "order" tables on "CustId" field

- Load joined data into "ordersummary" table using SCD Type 2

Data Transformations / Business Logic:

Step 1: Create "ordersummary" table if it does not exist

- Transformation Logic: Use `spark.sql()` to create table if it does not exist

Step 2: Join "customer" and "order" tables on "CustId" field

- Transformation Logic: Use `join()` to join tables on "CustId" field

Step 3: Load joined data into "ordersummary" table using SCD Type 2

- Transformation Logic: Use `applyChanges()` to apply SCD Type 2 changes to "ordersummary" table

Error Handling and Logging: Same as TR-DLT-001

TR-DLT-003: Update ordersummary Table on Customer Change

Related Functional Requirement(s): FR-DLT-003

Objective: Implement a Delta Live Tables pipeline to update "ordersummary" table whenever there is a change in the "customer" table.

Target Cluster Configuration: Same as TR-DLT-001

Source Data Details:

Customer Delta table: "customer", Delta format

Target Data Details:

ordersummary Delta table: "ordersummary", Delta format

Job Flow / Pipeline Stages:

- Identify changes in the "customer" table
- Update "ordersummary" table by making old records inactive and new records active
- Update StartDate and EndDate accordingly to maintain history

Data Transformations / Business Logic:

Step 1: Identify changes in the "customer" table

- Transformation Logic: Use `spark.read.format("delta").load()` to read "customer" table and identify changes

Step 2: Update "ordersummary" table by making old records inactive and new records active

- Transformation Logic: Use `applyChanges()` to apply changes to "ordersummary" table

Step 3: Update StartDate and EndDate accordingly to maintain history

- Transformation Logic: Use `withColumn()` to update StartDate and EndDate columns

Error Handling and Logging: Same as TR-DLT-001

TR-DLT-004: Create and Load customeraggregatespend Table

Related Functional Requirement(s): FR-DLT-004

Objective: Implement a Delta Live Tables pipeline to create and load "customeraggregatespend" table with aggregated data from "ordersummary" table.

Target Cluster Configuration: Same as TR-DLT-001

Source Data Details:

ordersummary Delta table: "ordersummary", Delta format

Target Data Details:

customeraggregatespend Delta table: "customeraggregatespend", Delta format

Job Flow / Pipeline Stages:

- Create "customeraggregatespend" table if it does not exist
- Aggregate "TotalAmount" column from "ordersummary" table and group by "Name" and "Date" columns
- Load aggregated data into "customeraggregatespend" table

Data Transformations / Business Logic:

Step 1: Create "customeraggregatespend" table if it does not exist

- Transformation Logic: Use `spark.sql()` to create table if it does not exist

Step 2: Aggregate "TotalAmount" column from "ordersummary" table and group by "Name" and "Date" columns

- Transformation Logic: Use `groupBy()` and `sum()` to aggregate data

Step 3: Load aggregated data into "customeraggregatespend" table

- Transformation Logic: Use `write.format("delta").saveAsTable()` to write to Delta table

Error Handling and Logging: Same as TR-DLT-001

TR-DLT-005: Implement using Delta Live Tables

Related Functional Requirement(s): FR-DLT-005

Objective: Implement the above requirements using Delta Live Tables.

Target Cluster Configuration: Same as TR-DLT-001

Job Flow / Pipeline Stages:

- Implement steps 1 to 4 using Delta Live Tables

Data Transformations / Business Logic: Same as TR-DLT-001 to TR-DLT-004

Error Handling and Logging: Same as TR-DLT-001